

Projet : Plateforme web de vente de fichiers STL et service d'impression 3D

Table des matières

	Contexte et objectif.....	1
	MERN = MongoDB + Express + React + Node.js.....	2
	Le MERN amélioré.....	2
	Public cible et positionnement.....	3
	Valeur ajoutée du projet.....	3
	Fonctionnalités.....	4
	Partie Utilisateur.....	4
	Partie Administrateur.....	5
	Architecture technique.....	5
	Front-end.....	5
	Back-end.....	5
	Performances et scalabilité.....	7
	Sécurité et sauvegardes.....	7
	Conformité RGPD.....	8
	Gestion des rôles et droits d'accès.....	8
	Hébergement envisagé.....	8
	Arborescence complète du site.....	8
	Pages principales.....	8
	Arborescence du projet.....	9
	Schéma MongoDB.....	10
	Détails du modèle User.....	10
	Diagramme UML (Cas d'utilisation).....	10
	Planification et livrables.....	12
	Planification du projet.....	12

Contexte et objectif

Le projet consiste à développer une **plateforme web complète** permettant :

- La **vente et l'achat** de fichiers STL destinés à l'impression 3D.
- La possibilité pour un utilisateur de **commander une impression 3D personnalisée**, soit à partir des fichiers proposés sur la plateforme, soit à partir de ses propres fichiers STL.
- Une **visualisation interactive** des modèles 3D directement dans le navigateur, grâce à une intégration de *Three.js*.

Le site repose sur une **architecture complète MERN améliorée** :

React + TypeScript + Vite (frontend) et Node.js + Express + MongoDB/MySQL (backend).

MERN = MongoDB + Express + React + Node.js

Lettre Composant		Rôle
M	MongoDB	Base de données NoSQL, pour stocker les fichiers, utilisateurs, commandes, etc.
E	Express.js	Framework minimal pour Node.js — gère les routes et la logique serveur (API REST).
R	React.js	Framework front-end (interface utilisateur) — permet de construire le site web interactif.
N	Node.js	Environnement JavaScript côté serveur — exécute Express et communique avec MongoDB.

Le MERN amélioré

Amélioration	Description	Ce que ça t'apporte
TypeScript	Ajoute un typage fort à React + Node	Moins d'erreurs, meilleure autocomplétion
Vite (au lieu de Create React App)	Build tool ultra rapide pour le front	Chargement instantané, hot reload
Architecture "modulaire"	/models, /controllers, /routes séparés	Code clair, scalable
Three.js intégré au front	Visualisation 3D directe	Expérience utilisateur moderne
Axios + API REST complète	Communication front ↔ back	Structure professionnelle
dotenv (.env)	Configuration propre (ports, URLs, secrets)	Sécurité et portabilité
Nodemon + ts-node	Serveur auto-reload en dev	Productivité accrue
Design responsive (CSS variables + grid)	Adapté mobile / desktop	UX pro sans frameworks lourds



Public cible et positionnement

La plateforme s'adresse principalement :

- aux **designers 3D indépendants** souhaitant vendre leurs créations STL,
- aux **entreprises** recherchant un service d'impression rapide et fiable,
- et aux **étudiants ou makers** désirant partager ou tester des modèles 3D.

Le positionnement de la plateforme repose sur :

- une **expérience utilisateur fluide et professionnelle**,
- une **infrastructure technique moderne**,
- et une **qualité de rendu des modèles 3D** supérieure grâce à la visualisation intégrée Three.js.



Valeur ajoutée du projet

Contrairement aux plateformes existantes comme [Cults3D](#) ou [Thingiverse](#), cette application vise une approche centrée sur :

- une **visualisation 3D intégrée** des fichiers STL, sans installation de logiciel tiers
- une **interface optimisée** pour la consultation, l'achat et la commande d'impressions 3D personnalisées
- une **infrastructure moderne et performante**, reposant sur un environnement **full-stack JavaScript (MERN amélioré)**
- un **système d'impression sur demande**, permettant d'envoyer son propre fichier STL pour fabrication,
- une **séparation claire des responsabilités** (front-end, API, stockage fichiers et base de données) facilitant la maintenance et la montée en charge
- une **architecture full-stack moderne (MERN améliorée)** garantissant des performances élevées et une maintenance facilitée.
- la possibilité de proposer à terme un **module de tarification automatisée** selon la complexité du modèle et la quantité demandée.

L'objectif est de fournir une plateforme **techniquement robuste, évolutive et orientée performance**, adaptée à un usage **professionnel** tout en restant accessible et fluide pour l'utilisateur final.

	Technologie du Web	
	Cahier des charges Projet fin d'année	

Fonctionnalités

Partie Utilisateur

- **Inscription / Connexion**
 - Gestion des comptes utilisateurs (mots de passe sécurisés, sessions JWT à venir).
- **Consultation du catalogue de fichiers STL**
 - Affichage des fichiers avec titres, catégories, prix et description.
 - Système de recherche par mot-clé et filtres par catégorie.
 - Données chargées dynamiquement depuis la base MongoDB.
- **Visualisation 3D en ligne des fichiers STL**
 - Aperçu 3D interactif grâce à **Three.js + STLLoader**.
 - Rotation, zoom et déplacement de la caméra (OrbitControls).
 - Recentrage et échelle automatique du modèle.
- **Téléchargement de fichiers STL achetés**
-  **Simulation de paiement**
 - **Le système de paiement sera simulé à l'aide d'une fausse API REST ou d'un environnement sandbox de Stripe, afin de reproduire un scénario d'achat réaliste sans transaction réelle.**
Les commandes seront ainsi enregistrées avec un statut "payée (simulée)" pour les tests de bout en bout.
- **Envoi de son propre fichier STL (Upload)**
 - L'utilisateur peut importer un fichier STL et ajouter des instructions pour demander une impression personnalisée.
 - Le fichier est stocké côté serveur (dans backend/public/files).
- **Suivi des commandes**
Statuts prévus : en attente, en cours, terminé.
- **Espace personnel utilisateur**
 - Historique des achats et téléchargements.
 - Liste des fichiers envoyés pour impression.

	Technologie du Web	 INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE
	Cahier des charges Projet fin d'année	

Partie Administrateur

- **Gestion des utilisateurs**
(création, modification, suppression — côté back + interface React Admin).
- **Gestion du catalogue STL**
 - Ajout de nouveaux fichiers STL (upload côté serveur).
 - Suppression et modification des fiches existantes.
 - Les fichiers sont automatiquement reliés à la base MongoDB.
- **Gestion des commandes d'impression 3D**
Interface prévue pour le suivi et la mise à jour du statut des commandes.
- **Statistiques**
nombre total de ventes, téléchargements, utilisateurs, fichiers.



Architecture technique



Front-end

- **Technologies** : React + TypeScript + Vite
- **Librairies principales** :
 - three (STLLoader, OrbitControls)
 - react-router-dom
 - axios (pour la communication API)
- **Structure des pages** :
 - Home, Catalogue, ProductDetail, Upload, Profile, Admin
- **Design** : sobre, sombre, responsive, avec une mise en page en grille et composants modulaires.



Back-end

- **Technologies** : Node.js + Express + TypeScript
- **Structure** :

```
backend/
  src/
    index.ts
```

	Technologie du Web	 INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE
	Cahier des charges Projet fin d'année	

```

routes/
  fileRoutes.ts
controllers/
  fileController.ts
models/
  File.ts
config/
  db_mongo.ts

```

Fonctionnalités back :

- API REST complète `/api/files` :
 - GET `/api/files` → liste des fichiers
 - GET `/api/files/:title` → détail d'un fichier
 - POST `/api/files/seed` → insertion de données de test
- Serveur statique pour les STL : `/files/<nom>.stl`
- **Base de données** : MongoDB (connexion locale ou Atlas)
- Modèle `File` :

```

{
  title: String,
  category: String,
  price: Number,
  description: String,
  downloads: Number,
  filename: String
}

```

⚙️ Schéma technique de l'architecture MERN améliorée

[Navigateur React + Three.js]

| requêtes HTTP (Axios)



[Serveur Node.js + Express (API REST)]

| —► MongoDB (stockage des fichiers, utilisateurs, commandes)

	Technologie du Web	
	Cahier des charges Projet fin d'année	

└─► Système de fichiers /public/files (stockage STL)

- Le **front-end React** communique via **Axios** avec l'API Express.
- L'API interagit avec **MongoDB** pour les données et sert les fichiers STL depuis le dossier /public/files.
- La configuration (ports, URL, clés) est centralisée dans un fichier `.env`.

⚡ Performances et scalabilité

- Le code front-end est optimisé via **Vite** pour un chargement instantané.
- L'API Node.js utilise un système de **cache** et des **requêtes asynchrones non bloquantes**.
- L'architecture MERN permet une **scalabilité horizontale** (ajout de serveurs sans refonte).
- Le stockage MongoDB est compatible avec **Mongo Atlas auto-clusterisé**, assurant la résilience du service.



Sécurité et sauvegardes

- Gestion des mots de passe (prévue avec `bcrypt` et `jsonwebtoken`).
- Validation des formulaires côté **client** (React) et côté **serveur** (Express).
- CORS activé pour la communication entre le front (: 5173) et le back (: 3000).
- Les mots de passe seront **hachés avec bcrypt** avant enregistrement.
- Les échanges front ↔ back se feront uniquement via **requêtes HTTPS** (en production).
- Les requêtes MongoDB sont sécurisées (aucune concaténation directe, utilisation de filtres typés).
- Les fichiers STL sont stockés dans un répertoire protégé, non modifiable depuis le front.
- Une **sauvegarde hebdomadaire automatique** de la base MongoDB sera mise en place (Mongo Atlas Backup).
- Les utilisateurs auront des **droits d'accès différenciés** (User / Admin) pour limiter les actions sensibles.



Conformité RGPD

- Les données personnelles (nom, email, mot de passe) sont stockées de manière sécurisée et chiffrée.
- Les utilisateurs disposent d'un droit de suppression et de rectification de leurs données.
- Aucune donnée sensible (moyen de paiement réel, adresse, etc.) n'est conservée en base.
- Les logs serveurs sont anonymisés et purgés automatiquement après 30 jours.
-



Gestion des rôles et droits d'accès

Le système de permissions distingue deux rôles principaux :

- **Utilisateur** : accès restreint à ses propres fichiers, commandes et profil.
- **Administrateur** : gestion complète du catalogue, des utilisateurs et des commandes.

Chaque opération critique (téléchargement, suppression, modification) est vérifiée par un middleware d'autorisation basé sur **JWT (JSON Web Token)**.



Hébergement envisagé

(à ajouter à la section Architecture technique — Backend)

- **Frontend** : déploiement prévu sur **Vercel** ou **Netlify**.
- **Backend** : hébergement sur **Render** ou **Railway.app** (serveur Node.js).
- **Base de données** : **MongoDB Atlas** (cluster cloud gratuit).
- Les fichiers STL seront hébergés dans le répertoire `/public/files` du backend, synchronisé avec le serveur.



Arborescence complète du site

Pages principales

Page	Description
Accueil	Présentation du service et mise en avant des fichiers populaires
Catalogue	Recherche, filtres et liste des fichiers STL

	Technologie du Web	 INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE
	Cahier des charges Projet fin d'année	

Page	Description
Détail fichier STL	Visualisation 3D + description + ajout au panier
Panier / Commande	Simulation d'achat
Uploader un fichier	Envoi d'un STL personnel pour impression
Espace utilisateur	Profil, historique d'achats et uploads
Administration	Gestion des fichiers, utilisateurs et commandes

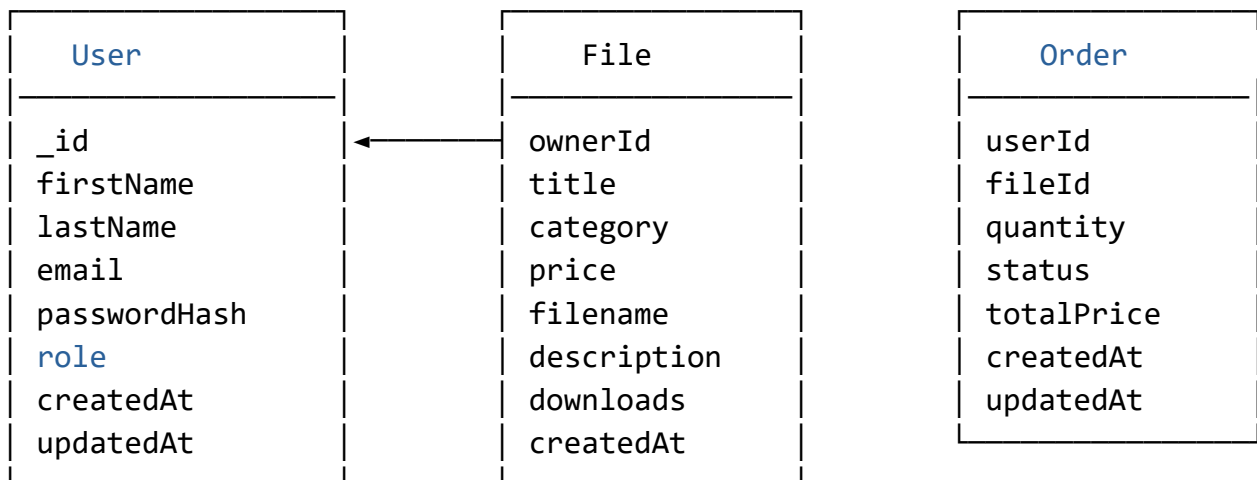
Arborescence du projet

```

stl-marketplace/
├── backend/
│   ├── src/
│   │   ├── config/db_mongo.ts
│   │   ├── controllers/fileController.ts
│   │   ├── models/File.ts
│   │   ├── routes/fileRoutes.ts
│   │   └── index.ts
│   ├── public/files/
│   │   ├── Boîte articulée.stl
│   │   ├── Support téléphone.stl
│   │   └── Vase paramétrique.stl
│   ├── package.json
│   └── .env
└── frontend/
    ├── src/
    │   ├── components/STLViewer.tsx
    │   ├── pages/{Home,Catalogue,ProductDetail,Upload,Profile,Admin}.tsx
    │   ├── lib/api.ts
    │   ├── styles/global.css
    │   └── App.tsx
    ├── package.json
    └── .env

```

Schéma MongoDB



Détails du modèle User

Champ	Type	Description
<code>_id</code>	ObjectId	Identifiant MongoDB unique
<code>firstName</code>	String	Prénom de l'utilisateur
<code>lastName</code>	String	Nom de famille
<code>email</code>	String (unique)	Sert d'identifiant de connexion
<code>passwordHash</code>	String	Mot de passe haché (bcrypt)
<code>role</code>	String ("USER" ou "ADMIN")	Gestion des droits
<code>createdAt, updatedAt</code>	Date	Suivi des inscriptions / modifications



Diagramme UML (Cas d'utilisation)

[Utilisateur]

└─> S'inscrire / Se connecter

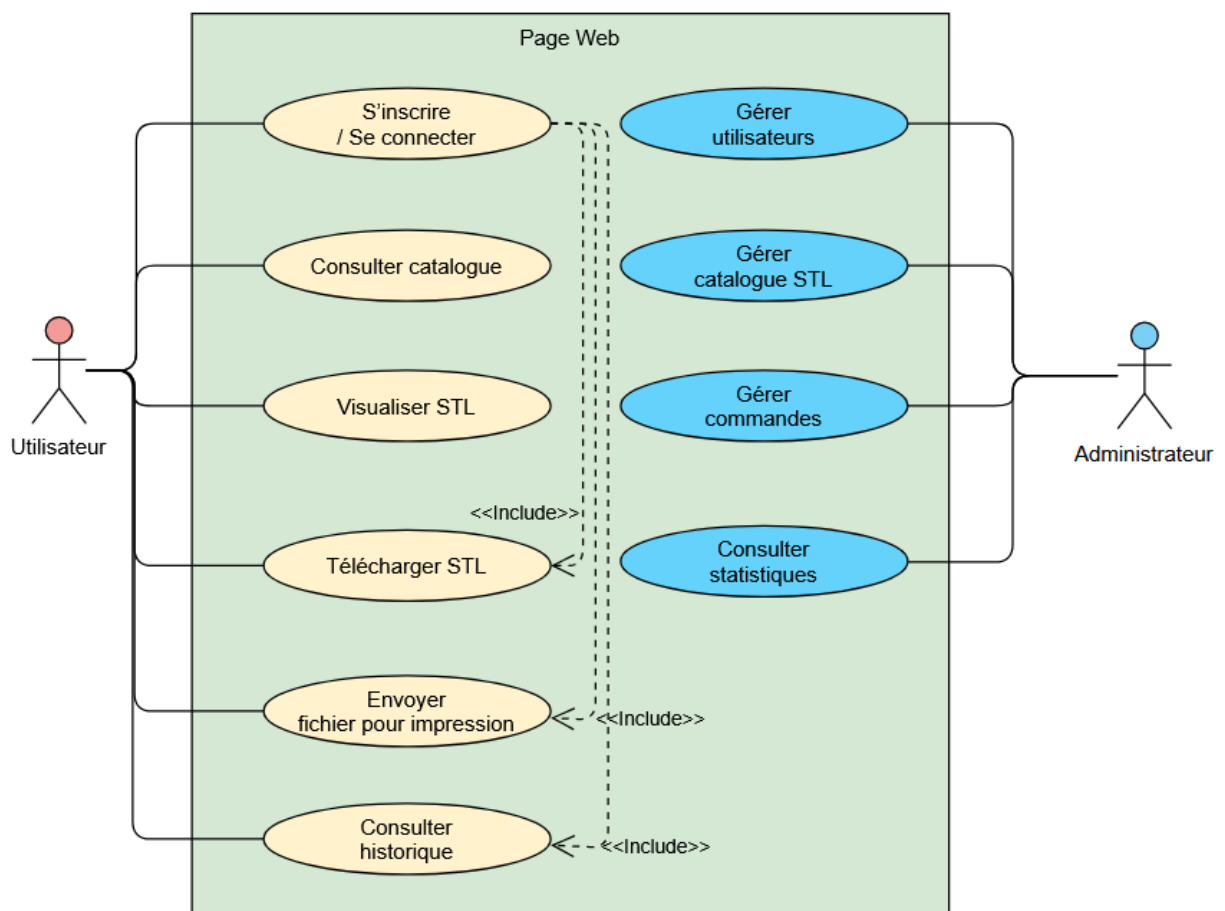
└─> Consulter catalogue

└─> Visualiser STL

- └─> Ajouter au panier
- └─> Télécharger STL
- └─> Envoyer fichier pour impression
- └─> Consulter historique

[Administrateur]

- └─> Gérer utilisateurs
- └─> Gérer catalogue STL
- └─> Gérer commandes
- └─> Consulter statistiques





Planification et livrables



Planification du projet

Phase	Période estimée	Objectifs principaux	Livrables	Critères de validation
Phase 1 – Initialisation	Semaine 1 à 3	Création de l'arborescence, configuration Vite/Express, base Mongo connectée	Serveur API + affichage catalogue	API /api/files renvoie les fichiers
Phase 2 – Visualisation 3D	Semaine 4 à 6	Intégration Three.js + affichage STL dynamique	Viewer 3D fonctionnel	Le modèle STL s'affiche correctement
Phase 3 – Upload et Auth	Semaine 7 à 9	Upload STL côté utilisateur + création comptes	Formulaire upload + CRUD fichiers	Les fichiers sont ajoutés et listés
Phase 4 – Paiement simulé et admin	Semaine 10 à 12	Simulation achat + interface admin	Dashboard admin	Toutes les routes CRUD admin fonctionnelles
Phase 5 – Finalisation	Semaine 13	Tests finaux, sécurité, rapport PDF	Documentation finale + rendu	Validation par tests utilisateurs